

Abstract

The purpose of this document is to give a terse outline of the most useful TikZ macros as a quick reference. At the time of writing it is nowhere near complete, but still may be useful for a quick reference for the basics.

Contents

1	Essentials	1
1.1	Rough idea of how TikZ works	1
1.2	Specifying coordinates	5
1.3	Path styling	5
1.4	Path decorations	5
1.5	Arrow Tips	6
1.6	Intersections	7
1.7	Node Shapes	8
2	Other	10
2.1	Layers	10
2.2	Tikzmark	10
2.3	Markings	11

1 Essentials

1.1 Rough idea of how TikZ works

When drawing a picture on a piece of paper, we may place our pen down and write in some places, while we may lift up our pen without drawing to move it to another location to draw there. “pen” while drawing, or they may move to another location on the page without drawing anything. It is with this idea that we write TikZ. TikZ is a language we use to instruct the computer how to move its “pen” around to draw diagrams.

A **path** is the basic unit of these instructions. Paths take the following form:

$$\backslash\text{path}\langle\textit{specification}\rangle;$$

A **specification** of a path is a sequence whose elements are separated by spaces consisting of graphics options and path operations.

A **graphics option** is a list of options in brackets, such as `[rounded corners]`, which may contain key-value pairs. Graphics options come in two forms.

graphics option	description	examples
local	applies only to the current path; can be scoped	[rounded corners], [draw], [fill]
global	applied to all path operations on the path; cannot be scoped	[red], [draw], [fill], [pattern], [shade], [clip], [use as bounding box]

A **path operation** tells TikZ a command that describes how the path is built. Path operations tell TikZ how the path is built. There are several types of path operations.

<code>\path ... <i><path operation></i> ... ;</code>	Description
<code>(B)</code>	set current position to (B)
<code>-- (B)</code>	straight line through (B)
<code>rectangle (B);</code>	rectangle
<code>-- (B);</code>	line
<code>circle (4);</code>	circle of radius 4
<code>ellipse(3 and 2);</code>	ellipse of width 6 and height 4
<code>arc($\theta_1:\theta_2:r$) (B);</code>	circular arc of radius r from angle θ_1 to θ_2
<code>arc($\theta_1:\theta_2:r_1$ and r_2)</code>	elliptical arc
<code>.. controls (C1) and (C2) .. (B);</code>	Bézier curve starting with tangent through C1 and entering B with tangent through C2
<code>grid (B);</code>	a grid
<code>parabola (B);</code>	parabola (several options for bending)
<code>sin (B);</code>	sine curve from (0,0) to $(\pi/2,1)$ stretched from the current position to (B)
<code>cos (B);</code>	cosine curve from (0,0) to $(\pi/2,1)$ stretched from the current position to (B)
<code>-- cycle;</code>	return to start and join up nicely

Table 2: Basic path operations.

The following is a more comprehensive list of path operations. (Note however that this is not even close to being an exhaustive list.)

<code>\path ... <i><path operation></i> ... ;</code>	Description
<code>\path ... path operation ... ;</code> <code>[<i><options for path operation></i>]</code>	description
Continued on next page	

<code>\path ... <path operation> ... ;</code>	Description
<code><coordinate></code>	move-to; sets (current subpath start) to <code><coordinate></code>
<code>--<coordinate or cycle></code> <code>rounded corners=<corner radius></code> <code>sharp corners</code>	line-to
<code>- <coordinate or cycle></code> <code>rounded corners=<corner radius></code> <code>sharp corners</code>	horizontal-then-vertical-to
<code> -<coordinate or cycle></code> <code>rounded corners=<corner radius></code> <code>sharp corners</code>	vertical-then-horizontal-to
<code>..controls <c> and <d>..<y or cycle></code> <code>rounded corners=<corner radius></code> <code>sharp corners</code>	curve-to; starts (resp. ends) with tangent through <code><c></code> (resp. <code><d></code>); if <code>and <d></code> is omitted, it is assumed <code><d> = <c></code>
<code>rectangle<corner></code> <code>rounded corners=<corner radius></code> <code>sharp corners</code>	rectangle; one corner at the current point and another corner at <code><corner></code>
<code>circle[<options>]</code> <code>radius=<value></code> <code>x radius=<value></code> <code>y radius=<value></code> <code>rotate=<angle></code> <code>scale=<scale factor></code>	circle; may also write <code>circ (<value>)</code> for a circle of radius <code><value></code>
<code>arc[<options>]</code> <code>start angle=<degrees></code> <code>end angle=<degrees></code> <code>delta angle=<degrees></code> <code>radius=<value></code> <code>x radius=<value></code> <code>y radius=<value></code>	arc; may also write <code>arc (<value>)</code> for an arc of radius <code><value></code>
<code>grid[<options>]<corner or cycle></code> <code>step=<number or dimension or coordinate></code> <code>xstep=<dimension or number></code> <code>ystep=<dimension or number></code> <code>help lines</code>	grid

Continued on next page

<code>\path ... <path operation> ... ;</code>	Description
<code>parabola[<options>](<bend</code> <code>nate>)<coordinate or cycle></code> <code> bend pos=<fraction></code> <code> parabola height=<dimension></code> <code> bend at start</code> <code> bend at end</code>	bend specifies where the bend should go. bend defaults to the old current point. For bend + (0,0), saying bend pos=0 (resp. bend pos=1) puts the bend at the old (resp. new) current point, or just put bend at start (resp. bend at end)
<code>sin<coordinate or cycle></code>	draws a (scaled and shifted) sine curve over $[0, \pi/2]$ from the old current point to the new current point
<code>cos<coordinate or cycle></code>	similar to sin
<code>to</code>	to path
<code>foreach<variables>[<options>] in <list></code> <code>{<path commands>}</code>	You can also write \foreach instead of foreach . E.g. <code>\foreach \x in {1,2,...,6} {\x, }</code> yields 1, 2, 3, 4, 5, 6, .
<code>let \p1 = <coordinate> in</code> <code>\x1</code> <code>\y1</code> <code>\p1</code> <code>{\veclen(\x1,\y1)}</code> <code>\path ... let \n1 = {sin(60)} in ...</code>	save a point's coordinates x-coordinate of point \p1 y-coordinate of point \p1 string containing coordinates of \p1 length of vector (x,y) save a number
scoping	When TikZ encounters an opening or a closing brace ({ or }) at some point where a path operation should come, it will open or close a scope . A scope is an area where local graphics options are applied only to that area. All options that can be applied “locally” will be scoped inside the scope. For example, if you apply a transformation like [xshift=1cm] inside the scoped area, the shifting only applies to the scope. On the other hand, an option like color=red does not have any effect inside a scope since it can only be applied to the path as a whole.
<code>node<foreachs>[<options>](<name>)</code> <code>(<coordinate>){<contents>}</code> <code> node contents=<contents></code> <code> behind path</code> <code> in front of path</code> <code> name=<node name></code>	Except for the described shifting, the coordinate system of a pic is reset prior to executing the pic type's code. This can be changed using the transform shape option, which applies “outer” transformation to the node. For at you should enclose <coordinate> in curly braces so that any comma inside <coordinate> does not confuse TeX.
<code>pic<foreachs>[<options>](<prefix>)</code> <code>(<coordinate>){<type>}</code> <code> pic type=<type></code> <code> transform shape</code>	The options and meanings are essentially identical to that of node . Inside of the pic type's code any mentioning of the origin refers to the last position used on the path or to the specified at .
Continued on next page	

1.2 Specifying coordinates

Name	Description
<code>\coordinate (B) at (3,5);</code>	name a point B

Coordinate	Description	Library Needed
<code>(x,y)</code>	Coordinate specification (Cartesian)	
<code>(\theta:r)</code>	Coordinate specification (polar)	
<code>++\langle coordinate \rangle</code>	Coordinate relative to previous position	
<code>+\langle coordinate \rangle</code>	Coordinate relative to previous position without saving the new position	
<code>(A B)</code>	(x -coord. of (A), y -coord. of (B))	
<code>(A -B)</code>	(x -coord. of (B), y -coord. of (A))	
<code>\$(A)+{\sin(60)}*(B)\$</code>	Coordinate calculations	calc
<code>\$(A)!.25!(B)\$</code>	0.25 the distance from (A) to (B)	calc
<code>\$(A)!3cm!(B)\$</code>	3 cm from (A) toward (B)	calc
<code>\$(A)!1.2!30:(B)\$</code>	Stretch by 1.2, then rotate by 30°	calc
<code>\$(A)!(B)!(C)\$</code>	Projection of point B onto $\overline{AC}$	calc
<code>\$(A)!(B)!30:(C)\$</code>	Project B onto $\overline{AC}$, then rotate 30°	calc

1.3 Path styling

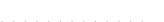
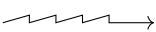
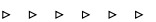
Thickness	Result	Dashing	Result
ultra thin		solid	
very thin		dotted	
thin		densely dotted	
semithick		loosely dotted	
thick		dashed	
very thick		densely dashed	
ultra thick		loosely dashed	
line width=5pt		dash pattern=on 2pt off 4pt	

Table 4: Styling path thickness and dashing. The code is `\tikz\draw[\langle style \rangle] (0,0) -- (2,0);`.

1.4 Path decorations

<code>\tikz\draw[->,decorate, decoration={...}] (0,0) -- (2,0);</code>	Result	Library
<code>-></code>		<code>decorations.pathmorphing</code>
<code>zigzag, post length=1mm</code>		<code>decorations.pathmorphing</code>
<code>random steps, amplitude=5mm</code>		<code>decorations.pathmorphing</code>
<code>saw, amplitude=1mm, post length=3mm</code>		<code>decorations.pathmorphing</code>
<code>bent, amplitude=1mm</code>		<code>decorations.pathmorphing</code>
<code>bumps, amplitude=1mm</code>		<code>decorations.pathmorphing</code>
<code>snake, amplitude=1mm</code>		<code>decorations.pathmorphing</code>
<code>brace, amplitude=5pt, aspect=0.5</code>		<code>decorations.pathmorphing</code>
<code>triangles</code>		<code>decorations.pathmorphing</code>
<code>coil, segment length=5pt, amplitude=5pt, aspect=0.5</code>		<code>decorations.pathmorphing</code>
<code>expanding waves, angle=7, amplitude=5pt,</code>		<code>decorations.pathmorphing</code>

1.5 Arrow Tips

Red text may be omitted. `arrows=<start arrow specification>-<end arrow specification>`

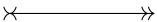
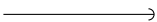
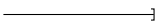
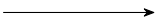
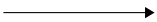
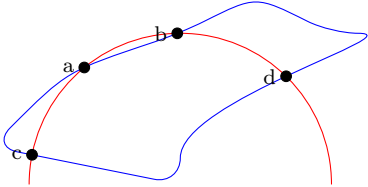
<code>\tikz\draw[<i>arrowhead</i>] (0,0)--(2,0);</code>	Result	Library
Continued on next page		
<code>-></code>		
<code>->></code>		
<code>><-></code>		
<code>->>></code>		
<code>->>>></code>		
<code>--{stealth}</code>		
<code>--{latex}</code>		
<code>--{to}</code>		
<code>--{Stealth}</code>		<code>arrows.meta</code>
<code>--{Arc Barb}</code>		<code>arrows.meta</code>
<code>--{Bracket}</code>		<code>arrows.meta</code>
<code>--{Hooks}</code>		<code>arrows.meta</code>
<code>--{Parenthesis}</code>		<code>arrows.meta</code>
<code>--{Straight Barb}</code>		<code>arrows.meta</code>
<code>--{Tee Barb}</code>		<code>arrows.meta</code>
<code>--{Classical TikZ Rightarrow}</code>		<code>arrows.meta</code>
<code>--{Computer Modern Rightarrow}</code>		<code>arrows.meta</code>
<code>--{Implies},</code> <code>double,</code> <code>double distance=1mm</code>		<code>arrows.meta</code>
<code>--{To}</code>		<code>arrows.meta</code>
<code>--{Latex[round]}</code>		<code>arrows.meta</code>
<code>--{Rectangle}</code>		<code>arrows.meta</code>
<code>--{Square}</code>		<code>arrows.meta</code>
<code>--{Stealth}</code>		<code>arrows.meta</code>
<code>--{Stealth[round]}</code>		<code>arrows.meta</code>
<code>--{Triangle}</code>		<code>arrows.meta</code>
<code>--{Turned Square}</code>		<code>arrows.meta</code>
<code>--{Circle[open]}</code>		<code>arrows.meta</code>

Table 7: Arrow tips [https://tikz.dev/tikz-\[\]arrows](https://tikz.dev/tikz-[]arrows).

1.6 Intersections

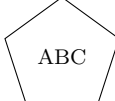
This requires `\usetikzlibrary{intersections}`.

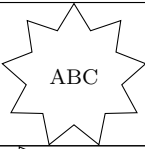
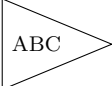
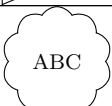
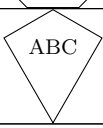
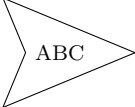
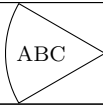
<pre> \begin{tikzpicture} \path[draw,red,name path=c1] (0,0) to[out=90,in=180] (2,2) to[out=0,in=90] (4,0); \path[draw,rounded corners=10pt, blue,name path=c2] (-0.5,0.5) .. controls +(1,1) .. ++(2.5,1.5) .. controls +(1,0.5) .. (4,2) to[out=0,in=90] (2,0)--cycle; \path[name intersections= {of=c1 and c2,by={a,b,c,d}}]; \foreach \p in {a,b,c,d} {\draw [fill=black] (\p) circle (2pt) node[left] {\p};} \end{tikzpicture} </pre>	
--	--

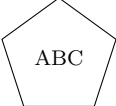
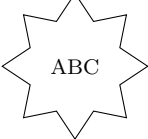
<code>\tikz\draw[<i><arrowhead></i>] (0,0)--(2,0);</code>	Result	Library Needed
---	---------------	-----------------------

Table 8: Arrow styles [https://tikz.dev/tikz-\[\]arrows](https://tikz.dev/tikz-[]arrows).

1.7 Node Shapes

<code>\tikz\node[draw,<i><node options></i>]{ABC};</code>	Result	Library
circle		
rectangle		
diamond		shapes.geometric
ellipse		shapes.geometric
regular polygon, regular polygon sides=5		shapes.geometric
Continued on next page		

<code>\tikz\node[draw,<node options>]{ABC};</code>	Result	Library
<code>star,</code> <code>star points=9</code>		<code>shapes.geometric</code>
<code>isosceles triangle</code>		<code>shapes.geometric</code>
<code>cloud,</code> <code>cloud puffs=10,</code> <code>cloud puff arc=120</code>		<code>shapes.symbols</code>
<code>forbidden sign</code>		<code>shapes.symbols</code>
<code>cylinder</code>		<code>shapes.geometric</code>
<code>semicircle</code> <code>shape border rotate=30</code>		<code>shapes.geometric</code>
<code>starburst,</code> <code>starburst points=30,</code> <code>starburst point height=0.2cm</code>		<code>shapes.symbols</code>
<code>tape,</code> <code>font=\fontsize{18}{18}\sffamily</code>		<code>shapes.symbols</code>
<code>cross out</code>		<code>shapes.misc</code>
<code>strike out</code>		<code>shapes.misc</code>
<code>rounded rectangle</code>		<code>shapes.misc</code>
<code>chamfered rectangle</code>		<code>shapes.misc</code>
<code>kite</code>		<code>shapes.geometric</code>
<code>dart</code>		<code>shapes.geometric</code>
<code>circular sector</code>		<code>shapes.geometric</code>
<code>cylinder,</code> <code>shape border rotate=90,</code> <code>aspect=0.25</code>		<code>shapes.geometric</code>
Continued on next page		

<code>\tikz\node[draw,<node options>]{ABC};</code>	Result	Library
<code>semicircle</code>		<code>shapes.geometric</code>
<code>regular polygon,</code> <code>regular polygon sides=5</code>		<code>shapes.geometric</code>
<code>star,</code> <code>star points=8</code>		<code>shapes.geometric</code>
<code>isosceles triangle</code>		<code>shapes.geometric</code>
<code>cloud,</code> <code>cloud puffs=10,</code> <code>cloud puff arc=120</code>		<code>shapes.symbols</code>
<code>forbidden sign</code>		<code>shapes.symbols</code>

2 Other

2.1 Layers

```

\pgfdeclarelayer{background}
\pgfdeclarelayer{foreground}
\pgfsetlayers{background,main,foreground} % (orders the layers)
\begin{pgfonlayer}{background}
\node {This node will appear on the background layer.};
\end{pgfonlayer}

```

2.2 Tikzmark

The following requires `\usetikzlibrary{tikzmark}`. Note the `overlay` option is essential is critical for this to work as shown.

<pre> Hello.\tikzmark{m}. ...content... \[\int_{\sum} q x d^2 \mu \to \int_{\tikzmark{n}} x \mu \] ...content...\tikzmark{q} \begin{tikzpicture}[overlay] \draw (pic cs:m) edge[-{Stealth},out=-45,in =225] (pic cs:n); \draw (pic cs:m) edge[-{Implies},double, double distance=5pt, out=45,in=45,looseness=2] (pic cs:q); \draw[font={\sffamily,\bfseries},align=left, ultra thick,red,decorate,decoration={brace, amplitude=10pt, aspect=0.5},very thick] ([xshift=2cm]pic cs:m) to["Hello World!\n How are you?" above=0.5cm,sloped] ([xshift=2cm]pic cs:q); \draw[blue,xshift=0.5] (pic cs:m) -- (pic cs: q); \end{tikzpicture} </pre>	<pre> Hello.. ...content... ...content... ...content... ...content... ...content... ...content... $\int_m \sum q x d^2 \mu \rightarrow \int_x \mu$... content content content content content content ... </pre>
---	--

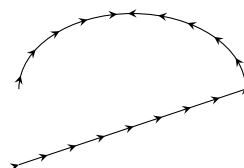
2.3 Markings

The following requires `\usetikzlibrary{decorations.markings}`.

```

\begin{tikzpicture}[postaction={
  decorate,decoration={markings,
    mark=between positions 0 and .75
    step 4mm with {\arrow{stealth}},
    mark=between positions .75 and 1
    step 4mm with {\arrowreversed{
    stealth}}}}}
\draw (0,0) -- (5,1) arc
  (0:180:3.5 and 2);
\end{tikzpicture}

```



Excerpt from 14.22 of PGF manual: Interacting with the Soft Path subsystem

During construction TikZ stores the path internally as a soft path. Sometimes it is desirable to save a path during the stage of construction, restore it elsewhere and continue using it. There are two keys to facilitate this operation, which are explained below. To learn more about the soft path subsystem, refer to section 121 . (no default) Save the current soft path into $\langle \text{macro} \rangle$. `/tikz/use path= $\langle \text{macro} \rangle$` (no default) Set the current path to the soft path stored in $\langle \text{macro} \rangle$.

```

\usetikzlibrary {intersections}
\begin{tikzpicture}
  \path[save path=\pathA,name path=A] (0,1) to [bend left] (1,0);
  \path[save path=\pathB, name path=B]
    (0,0) .. controls (.33,.1) and (.66,.9) .. (1,1);
  \fill[name intersections={of=A and B}] (intersection-1) circle (1pt);
  \draw[blue][use path=\pathA];
  \draw[red] [use path=\pathB];
\end{tikzpicture}

```

This gives 